

Ecuación de Poisson

Consideremos el problema de valores a la frontera para la ecuación de Poisson

$$-u''(x) = f(x), \quad x \in (0, L), \quad u(0) = u(L) = 0. \quad (1)$$

La solución se puede escribir como una integral usando la función de Green $G(x, \xi)$:

$$u(x) = \int_0^L G(x, \xi) f(\xi) d\xi,$$

donde la función de Green está dada por:

$$G(x, \xi) = \begin{cases} \frac{x(L - \xi)}{L}, & \text{si } x \leq \xi, \\ \frac{\xi(L - x)}{L}, & \text{si } x > \xi. \end{cases}$$

Entonces, la solución general del problema es:

$$u(x) = \int_0^x \frac{\xi(L - x)}{L} f(\xi) d\xi + \int_x^L \frac{x(L - \xi)}{L} f(\xi) d\xi. \quad (2)$$

Ejercicios

1. Implementa la cuadratura de la regla del punto medio compuesta para discretizar el problema (2) suponiendo que $L = 1$.

Solución. definimos una h la cual es

$$h = \frac{b - a}{n}$$

donde n son las particiones del espacio, la cuadratura de la regla del punto medio compuesta esta definida como

$$x_i^* = a + \left(i - \frac{1}{2}\right) h$$

donde x_i^* son los puntos de evaluación y mi integral es

$$\int_a^b f(x) dx \approx h \sum_{i=1}^n f(x_i^*)$$

entonces mi discretización de mi ecuación es

la evaluación de ξ

$$\xi_i^* = 0 + \left(i - \frac{1}{2}\right) h$$

donde n_1 y n_2 son N donde n_1 son los nodos del dominio de $[0, x]$ y n_2 los nodos del dominio $[x, L]$ donde u es

$$u(x) \approx \sum_{i=1}^{n_1} \frac{\xi_i^*(L-x)}{L} f(\xi_i^*)h + \sum_{i=n_1}^{n_2} \frac{x(L-\xi_i^*)}{L} f(\xi_i^*)h$$

para problema específico donde $L = 1$

$$u(x) \approx \sum_{i=1}^{n_1} \xi_i^*(1-x) f(\xi_i^*)h + \sum_{i=n_1}^{n_2} x(1-\xi_i^*) f(\xi_i^*)h$$

donde se considera que h es uniforme, donde $h = 1/(n+1)$ ahora si consideramos que se discretizar también respecto de x , entonces vamos tener dato de u , respecto a cada j -ésimo valor de x , vamos tener un vector $\mathbf{x}$ y de $\mathbf{u}$ y un vector de $\mathbf{f}$ esto se puede ver como sistema lineal como

$$\mathbf{u}(\mathbf{x}) \approx A\mathbf{f}$$

donde A es

$$[A]_{ij} = \begin{cases} \xi_i^*(1-x_j)h & x_j \leq \xi_i \\ x_j(1-\xi_i^*)h & x_j \geq \xi_i \end{cases}$$

donde $\mathbf{f}$ es

$$[\mathbf{f}]_i = f(\xi_i)$$

donde $\mathbf{f}$ es mi incógnita ■

2. Haz un modelo a priori de $f(x)$ mediante un Gaussian Markov Random Field usando el artículo de Bardsley como referencia. Considera que la condición de frontera de f es conocida. Nota: Al construir la distribución a priori, es necesario añadir un offset (desplazamiento) a la media; de lo contrario, la media a priori será cero y el modelo no capturará adecuadamente la información disponible sobre f .

Solución. Se puede considerar que nuestro es un problema inverso lineal

$$b = Ax$$

donde $b \in \mathbb{R}^m$ que son los datos observados, donde x es un vector n vector desconocido y A es una matriz de mn es modelo . en la practica tenemos ruido.

$$b = Ax + \eta$$

donde η es un vector random gaussiano. mi problema es una ecuacion Poisson

$$b = Poiss(Ax + \gamma)$$

donde γ es un vector m de recuentos de fondo y se supone conocido.

$$p(b|x) \propto \exp\left(-\frac{\lambda}{2}\|Ax - b\|^2\right)$$

su forma logaritmica es

$$\log p(b|x) \propto -\frac{\lambda}{2} \|Ax - b\|^2$$

la probabilidad posteriori funcion densidad $p(x|b)$ se puede escribir como

$$p(x|b) \propto p(b|x)p(x)$$

su forma logaritmica es

$$\log p(x|b) \propto \log p(b|x) + \log p(x)$$

el modelo a priori que proponen en el artículo de bardsley es

$$P(x) \propto \exp \left(-\frac{\delta}{2} x^T L x \right)$$

pero para este trabajo vamos a añadir una media μ

$$P(x) \propto \exp \left(-\frac{\delta}{2} (x - \mu)^T L (x - \mu) \right)$$

donde δL es matriz covarianza inversa. donde vector $\mathbf{x} = (x_1, \dots, x_n)$ y donde $\mathbf{x}_{-i} = (x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n)$

$$\mathbf{x}|\mathbf{x}_{-i} \sim N \left(\sum_{j \neq i} \beta_{ij} x_j, \kappa_i^{-1} \right)$$

los parametros satisfacen $\beta_{ij}\kappa_i = \beta_{ji}\kappa_j$ para todo i y j

$$p(x) = (2\pi)^{-n/2} |Q|^{1/2} \exp \left(-\frac{1}{2} (x - \mu)^T Q (x - \mu) \right)$$

su forma logaritmica es

$$\log p(x) = -\frac{n}{2} \log(2\pi) + \frac{1}{2} \log |Q| - \frac{1}{2} (x - \mu)^T Q (x - \mu)$$

donde $|Q|$ es el $\det(Q)$ es

$$\det |Q| = \prod_{i=1}^n \lambda_i$$

si calculamos su logaritmo

$$\log \det |Q| = \sum_{i=1}^n \log \lambda_i$$

donde

$$[Q]_{ij} = \begin{cases} \kappa_i, & i = j \\ -\kappa_i \beta_{ij} & i \neq j \end{cases}$$

La malla computacional que se utiliza para realizar la discretización numérica define un conjunto de nodos i y un sistema de vecindad ∂_i para $i = 1, \dots, n$. Resulta que los elementos de ∂_i son los nodos $j = i$ para los cuales $[Q]_{ij} = 0$, y por lo tanto,

$$\partial_i = \{j \neq i | \beta_{ij} \neq 0\}$$

además, si $\mathbf{x}_{\partial_i} = \{x_j | j \in \partial_i\}$, entonces $p(x_i | x_{-\partial_i}) = p(x_i | x_{\partial_i})$. Como ejemplo, supongamos que

$$\kappa_i = \frac{\delta n_i}{h^2},$$

donde $n_i = |\partial_i|$ y $\delta > 0$, y

$$\beta_{ij} = \begin{cases} \frac{1}{n_i}, & \text{si } j \in \partial_i, \\ 0, & \text{en otro caso.} \end{cases}$$

Entonces, toma la forma

$$x_i | x_{\partial_i} \sim \mathcal{N}\left(\bar{x}_{\partial_i}, \left(\frac{\delta n_i}{h^2}\right)^{-1}\right),$$

donde

$$\bar{x}_{\partial_i} = \frac{1}{n_i} \sum_{j \in \partial_i} x_j$$

es el promedio de los valores vecinos, y la función de densidad conjunta está dada por

$$[Q]_{ij} = \frac{\delta}{h^2} \begin{cases} n_i, & i = j, \\ -1, & j \in \partial_i, \\ 0, & \text{en otro caso.} \end{cases}$$

■

3. Implementa la solución bayesiana del problema inverso de determinar

$$f(x) = x$$

dadas observaciones ruidosas de

$$u(x) = \frac{1}{6}x(1-x)(1+x)$$

con ruido aditivo gaussiano con relación de señal a ruido 100.

Solución.

■

4. Calcula la media y la covarianza posterior de forma analítica.
5. Muestra tu distribución posterior y compara la media posterior numérica tomando en cuenta el tiempo integrado de autocorrelación.
6. Calcula la constante de normalización de manera analítica, y numéricamente de la manera siguiente:

(a) Usa la función `scipy.stats.gaussian_kde` para ajustar una distribución gaussiana $G(f)$ a las muestras que obtuviste.

(b) Aproxima

$$\begin{aligned} z_0 &= \int \pi(u|f)\pi(f) df \\ &= \int \frac{\pi(u|f)\pi(f)}{G(f)} G(f) df \\ &\approx \frac{1}{N} \sum_{j=1}^N \frac{\pi(u|f_j)\pi(f_j)}{G(f_j)} \end{aligned}$$

donde $\{f_j\}_{j=1}^N$ son muestras de $G(f)$.

El Truco del Log-Sum-Exp: Una Herramienta para la Estabilidad Numérica

Motivación

En muchos cálculos probabilísticos, frecuentemente nos encontramos con expresiones del tipo:

$$\log \left(\sum_{i=1}^N e^{a_i} \right)$$

Esta expresión aparece naturalmente al calcular el logaritmo de una mezcla, log-verosimilitud de modelos marginales, y en particular al estimar esperanzas en espacio logarítmico como:

$$\log \left(\frac{1}{N} \sum_{i=1}^N w_i \right) = \log \left(\frac{1}{N} \sum_{i=1}^N e^{\log w_i} \right)$$

El cálculo directo de esta cantidad puede sufrir de inestabilidad numérica debido a desbordamientos (overflow) o subdesbordamientos (underflow) cuando los exponentes a_i son muy positivos o muy negativos.

El Truco

Sean $a_1, a_2, \dots, a_N \in \mathbb{R}$. Definimos:

$$M = \max_i a_i$$

Entonces:

$$\log \left(\sum_{i=1}^N e^{a_i} \right) = M + \log \left(\sum_{i=1}^N e^{a_i - M} \right)$$

¿Por qué Funciona?

Dado que $a_i - M \leq 0$, los exponenciales $e^{a_i - M}$ son todos menores o iguales a 1, evitando desbordamientos. Además, si los a_i son muy negativos, restar M lleva el mayor término a cero, evitando subdesbordamientos en los otros términos.

Ejemplo

Supongamos:

$$a = [-1000, -1001, -999]$$

Entonces:

$$\sum_{i=1}^3 e^{a_i} = e^{-1000} + e^{-1001} + e^{-999}$$

Estos números son demasiado pequeños para ser representados en punto flotante estándar. Usando el truco:

$$\log \left(\sum_{i=1}^3 e^{a_i} \right) = -999 + \log \left(e^{-1} + e^{-2} + 1 \right) \approx -999 + \log \left(1 + e^{-1} + e^{-2} \right)$$

En la Práctica

En Python (por ejemplo, usando NumPy), esto se implementa como:

```
def logsumexp(a):
    a_max = np.max(a)
    return a_max + np.log(np.sum(np.exp(a - a_max)))
```

Muchas bibliotecas científicas como `scipy` y `PyTorch` también implementan una función estable llamada `logsumexp`.